17 宽阶段、动态树与几何缓存

> 目标：从不会漏对的双重循环出发，理解 Box2D 为什么维护三棵树、为什么允许包围盒变胖，以及三种“缓存”究竟省掉了什么。
>
> 本章对应 Box2D 3.2.0，提交 `dc67c40ba2ce1ad384b0c985dbc1b539cba9221b`。 下列次数和几何数字是理论推导或手算，不是本次运行得到的性能测量。

17.1 先保留一份朴素正确的答案

暂时不考虑动态树。 对所有不同形状对执行过滤和 AABB 相交测试，通过后进入窄阶段。

教学伪代码，非源码：

```
for i in [0, shapeCount):
    for j in (i, shapeCount):
        if allowedPair(i, j) and overlaps(aabb[i], aabb[j]):
            narrowPhase(i, j)
```

第二层从 `i + 1` 开始，同时避免自己碰自己和重复检查。 它很慢，却非常适合充当后续优化的正确性参考。

令 n 为形状数量，N 为候选对检查次数，两者都是无量纲计数：

$$N = \frac{n(n-1)}{2}.$$

形状数 n	每帧候选对数 N
100	4,950
1,000	499,500
10,000	49,995,000

若每秒推进 60 帧，最后一行意味着每秒约 29.997 亿次候选检查。 这还没有计算多边形距离、接触点和约束求解。

第一项优化目标因此不是“把一次碰撞公式写得更短”，而是减少不可能碰撞的形状对。 但必须牢记输出规模的下界：如果所有包围盒真的互相重叠，候选结果本身仍可能达到 $O(n^2)$。 动态树不能把必须输出的五千万个结果变成十万个。

17.2 过滤规则首先是语义

在优化过滤之前，先写对过滤。 `b2ShouldShapesCollide` 的规则不是把所有条件做一次逻辑与。 它先处理组号，再处理双向掩码。

```
static inline bool b2ShouldShapesCollide( b2Filter filterA, b2Filter filterB )
{
	if ( filterA.groupIndex == filterB.groupIndex && filterA.groupIndex != 0 )
	{
		return filterA.groupIndex > 0;
	}

	return ( filterA.maskBits & filterB.categoryBits ) != 0 && ( filterA.categoryBits &
filterB.maskBits ) != 0;
}
```

按下面的顺序判断：

1. 两个 `groupIndex` 相等且非零：正值允许，负值禁止。
2. 其余情况：A 的掩码接受 B 的类别，而且 B 的掩码也接受 A 的类别。

A	B	这个过滤函数的结果
`category=1, mask=2, group=0`	`category=2, mask=1, group=0`	允许
`category=1, mask=2, group=0`	`category=2, mask=4, group=0`	禁止
`mask=0, group=3`	`mask=0, group=3`	允许
`mask=全部, group=-3`	`mask=全部, group=-3`	禁止
`group=3`	`group=4`	回到双向掩码判断

第三行容易被“提前按 mask 剪枝”的优化误杀。 因此当前宽阶段查询树时使用 `B2_DEFAULT_MASK_BITS`，把这项形状过滤留给回调。 不能直接拿查询形状的 `maskBits` 替换它。

正组号也不是绝对的“无条件碰撞通行证”。 它只覆盖这个函数中的掩码规则。 同一刚体上的形状、传感器处理路径、没有支持算法的形状组合、关节禁止碰撞和自定义过滤，仍有各自的判断。

`b2PairQueryCallback` 的主要顺序是：

1. 排除自身代理与重复的移动代理对。
2. 查询 `pairSet`，已有接触对不重复创建。
3. 排除同一刚体和传感器。
4. 执行形状过滤和形状类型支持检查。
5. 检查刚体之间的关节过滤。
6. 必要时执行用户自定义过滤。
7. 把新候选写入该查询自己的结果链表。

实验：用表中的五组输入测试过滤；尤其检查“同正组号、双方 mask 为零”。 优化前后应得到完全相同的允许集合，而不是仅比较接触总数。

17.3 为什么是三棵树

朴素方案可以把所有形状放在同一棵树里，查询后再根据刚体类型过滤。 这样正确，但会访问不少永远不产生实体接触的组合。

当前宽阶段按 `static`、`kinematic`、`dynamic` 分开存放代理。 查询策略由被移动代理的类型决定：

被查询的移动代理	查询哪些树
动态	运动学、静态、动态
运动学	动态
静态	动态

所以“移动动态物体查询另外两棵树”是不完整的。 它还要查询动态树，因为动态物体之间也会碰撞。

“移动集合”也不等于“速度不为零的动态刚体”。 静态形状修改、创建策略等操作，也可能让静态代理进入移动集合。

重复对有两种不同来源：

- A 和 B 都移动，分别查询时可能都发现对方。
- A 和 B 已经有持久接触，本帧查询又发现一次。

前一种使用移动位图与代理键规则去重。 后一种使用 `pairSet` 检查已有形状对。 不要把这两个用途合并成一句“用哈希缓存所有碰撞”。

代价：三棵树增加维护结构，但让类型规则更早减少无效查询。 是否值得，取决于场景中三类代理的数量和移动比例。

实验：分别创建大量静态地形、运动学平台和动态箱子。 固定同一移动序列，记录各类查询访问的树及候选集合，与朴素类型过滤后的全配对结果比较。 这与 CCD 的目标树选择不是同一套流程，不要混用两者规则。

17.4 胖 AABB：允许小移动暂时不维护树

如果每移动 1 mm 就删除并重新插入代理，缓慢滑动的物体也会持续支付树维护成本。 胖 AABB 的思想是预留一点空间：只要新包围盒仍在旧预留范围内，就不调整树代理。

这里要区分两层范围：

- `shape->aabb` 已包含 speculative distance，不一定是紧贴几何的盒。
- `shape->fatAABB` 在它外面再加用于减少树维护的 margin。

在默认米制单位下：

量	默认值
`B2_LINEAR_SLOP`	0.005 m

量	默认值
`B2_SPECULATIVE_DISTANCE`	0.020 m
`B2_MAX_AABB_MARGIN`	0.050 m
`B2_AABB_MARGIN_FRACTION`	0.125

`b2ComputeShapeMargin` 根据形状尺度计算 margin，再限制最大值。 若圆半径为 r，单位 m，则圆的 margin 为：

$$m = \min(0.05, 0.125r).$$

例如 $r = 0.1\ \mathrm{m}$，则 $m = 0.0125\ \mathrm{m}$，不是固定 5 cm。 其他形状使用代码中对应的尺度计算，不能统一把它替换为刚体半径。

再看一个 margin 已达到 0.05 m 的盒子。 假设几何范围为 $[0,1] \times [0,1]\ \mathrm{m}$：

1. 加 speculative distance 后，范围为 $[-0.02, 1.02]^2\ \mathrm{m}$。
2. 再加树维护 margin 后，范围为 $[-0.07, 1.07]^2\ \mathrm{m}$。
3. 向右移动 0.04 m，新带 speculative 范围的右边界为 1.06 m，仍被包含。
4. 向右累计移动 0.051 m，右边界变成 1.071 m，越过原 fat AABB。

当前 `b2FinalizeBodiesTask` 对非 fast 路径重新计算 `aabb`。 只有 `b2AABB_Contains(shape->fatAABB, aabb)` 为假，才生成新 fat AABB 并设置 enlarged 标记。 静态代理初始化还有自己的额外 margin 分支，不能把上述动态例子当成所有类型统一规则。

代价：盒子更胖，会产生更多“粗测重叠、细测并未接触”的候选。 margin 太小使树维护频繁；太大则让候选和持久接触数量增加。 这是一项时间成本交换，不是越大越快。

实验：让盒子每帧右移 1 mm，记录 containment 失败次数。 再插入一次 51 mm 跳跃，检查代理确实更新。 同时使用朴素几何相交检查确认没有遗漏真实相交对。

17.5 动态树如何排除整片空间

动态树的叶节点对应代理。 内部节点的 AABB 包住两个子节点，类别位则是两个子节点类别位的按位或。

如果查询盒与某个内部节点都不相交，其全部后代都可跳过。 与逐叶测试相比，一次判断可能排除几百个代理。

`b2DynamicTree_Query` 使用显式栈遍历：

1. 根节点入栈。
2. 弹出一个节点，增加 `nodeVisits`。
3. AABB 和类别位都通过才继续。
4. 遇到叶节点调用回调，增加 `leafVisits`。
5. 内部节点把两个孩子入栈。

当前先压 `child1`，再压 `child2`，所以 LIFO 弹出顺序是 `child2` 先。 这个顺序会影响候选链表顺序，不能认为只要树集合相同，任意遍历次序都无关紧要。

理想平衡的 1,024 叶树深度约为 10。 但“深度 10”不代表每次查询只访问 10 个节点。 查询可能同时进入多个分支，实际成本应观察 `nodeVisits` 和 `leafVisits`。

实验：对固定查询集合，同时记录树高和访问数。 比较均匀分布与全部挤在一处的场景，观察树高相近时查询工作量也可能相差很大。

17.6 插入位置为什么比较周长

最简单的插入方案是任意选一个叶子作兄弟。 它可以保持树结构合法，却可能迅速制造很大的内部包围盒。

当前 `b2FindBestSibling` 用周长代价选择兄弟。 源码中有些变量叫 `area`，实际调用的是 `b2Perimeter`。 在二维这里比较的是长度，单位 m，而不是面积 m²。

令 $P(X)$ 为盒 X 的周长：

$$P(X) = 2(\mathrm{width}(X) + \mathrm{height}(X)).$$

令 D 为待插入叶盒、N 为当前节点盒、I 为祖先累计增长代价，三种代价的单位均为 m：

$$C_{\mathrm{current}} = P(N \cup D) + I,$$

$$I' = I + P(N \cup D) - P(N).$$

若孩子 C 是叶子，候选代价为 $P(C \cup D) + I'$。 若孩子是内部节点，代码使用下界：

$$L_C = I' + P(C \cup D) + \min(P(D) - P(C), 0).$$

这个下界用于判断继续下降是否还有希望改进结果。 算法沿一条贪心路径下降，不是遍历所有叶子求全局最优。 内部孩子下界相同时，还会使用中心距离平方打破平局。

手算一次插入

设两个旧叶子为：

- $A = [0, 2] \times [0, 2]\ \mathrm{m}$。
- $B = [8, 10] \times [0, 2]\ \mathrm{m}$。
- 根盒为 $[0, 10] \times [0, 2]\ \mathrm{m}$，周长 24 m。

插入 $D = [2.5, 3.5] \times [0, 1]\ \mathrm{m}$：

候选	并集周长	继承代价	总代价
根	24	0	24
A	11	0	11
B	19	0	19

因此选择 A 作兄弟。 新建父节点包住 A 和 D，随后向上更新盒、类别位和高度。

如果把 D 改成 $[11, 12] \times [0, 1]\ \mathrm{m}$，根并集周长变成 28 m。 孩子继承的增长代价为 $28 - 24 = 4\ \mathrm{m}$。 此时 A 候选为 $28 + 4 = 32\ \mathrm{m}$，B 候选为 $12 + 4 = 16\ \mathrm{m}$，所以选择 B。

`b2InsertLeaf` 还必须处理数组扩容后节点指针失效的问题。 稳定节点索引不意味着先前保存的 `nodes` 指针仍然有效。

17.7 旋转与重建分别修复什么

插入启发式不能保证树永远优良。 物体移动之后，原本相近的一组叶子可能已经相距很远。

局部旋转尝试交换孩子与孙子，让内部盒的周长更小。 这不是只比较高度差的 AVL 旋转。

考虑 `b2RotateNodes` 中 B 是叶、C 是内部节点的分支。 C 的孩子为 F 和 G，所有盒的 y 范围均为 $[0, 1]\ \mathrm{m}$：

盒子	x 范围
B	[0,1]
F	[10,11]
G	[2,3]

原 C 包住 F 和 G，周长为 $2(9 + 1) = 20\ \mathrm{m}$。 交换 B 与 F 后，C 包住 B 和 G，周长为 $2(3 + 1) = 8\ \mathrm{m}$。 交换 B 与 G 后，C 包住 B 和 F，周长为 $2(11 + 1) = 24\ \mathrm{m}$。 所以第一种交换把局部代价从 20 m 降到 8 m，根范围并未改变。

边界细节也值得读：一叶分支只有旧代价严格小于两个候选时才直接返回，因此等成本时仍可能交换。 两子均为内部节点的分支，则用严格小于当前最优代价来选交换。

当前代理操作并不都旋转：

操作	主要行为
`b2DynamicTree_CreateProxy`	插入时允许旋转
`b2DynamicTree_MoveProxy`	删除再插入，但 `shouldRotate=false`
`b2DynamicTree_EnlargeProxy`	扩大祖先盒并标记，不立即重插

重建处理的是更大范围的结构质量。 `b2DynamicTree_Rebuild(tree, false)` 把未扩大子树整体保留为重建单元；扩大部分则拆开重组。 `fullBuild=true` 才展开全部内部结构。 返回的 `leafCount` 是本次重建单元数量，局部重建时不必等于代理总数。

当前启用的 `B2_TREE_HEURISTIC` 为 0。 `b2PartitionMid` 按中心范围最长轴的**空间中点**划分，不是对象数量中位数，也不是全局 SAH 排序。 例如中心 x 为 0、1、8、9，y 相同，中点 4.5 把它们分成两组。 如果中心全部相同而划分失败，则退回 `count / 2`。

宽阶段后台重建任务只重建动态树和运动学树，且可与窄阶段重叠执行。 修改树的 refit 阶段之前必须等待这个任务完成。

代价：重建需要扫描、临时数组和可能的分配。 树高降低也不等于整帧耗时必然降低，应同时测查询访问数和重建开销。

实验：复用 `TreeRebuildAndValidateTest` 的固定代理布局。 保存相同查询的结果集合、访问数和树高，分别在重建前后比较。 正确性要求集合相同，性能结论则来自实测，不能预填一个加速倍数。

17.8 三种缓存不能混为一谈

缓存	保存什么	主要省掉什么
接触点 ID 热启动	点特征 ID 对应的旧冲量	求解器从零建立支撑冲量的工作
GJK simplex cache	支持点索引组成的单纯形	距离搜索的部分迭代
Contact recycling	相对姿态及已有 manifold	满足条件时整次接触更新

接触点 ID 热启动

朴素求解每帧把冲量从零开始。 稳定堆叠却往往需要重新找回与上一帧相近的支撑冲量。

当前 `b2UpdateContact` 仍重新计算 manifold，再按新旧点的特征 ID 匹配冲量。 假设旧点为 `ID7: 1.2 N·s`、`ID9: 0.8 N·s`，新点为 `ID9`、`ID12`。 新初值应为 0.8 N·s 和 0，而不是按数组位置复制 1.2 和 0.8。

未匹配点从零开始。 源码中把未匹配旧冲量重新分配的尝试位于 `#if 0`，当前并不执行。 这项优化不等于跳过窄阶段。

实验：记录点 ID 与新旧冲量，制造接触点交换顺序和点消失的情形，检查匹配的是身份而非槽位。

GJK 单纯形缓存

朴素 GJK 每次从空单纯形开始。 连续小移动时，上次靠近最近点的支持点索引可能仍是好的起点。

`b2MakeSimplexFromCache` 根据缓存的 `indexA/indexB` 从**当前**代理中重新取点。 它并不是把旧世界空间顶点原封不动拿来继续算。 空缓存则从索引 0 的支持点对开始。

缓存依赖对应的几何索引仍有效；几何拓扑变化不能盲目复用。 也不是所有形状对的 manifold 算法都使用 GJK。

实验：固定两个凸形状的小位移序列，比较每次清空 cache 与连续使用 cache。 检查距离、最近点及迭代总数，不能事先声称每次必少三轮。

17.9 Contact recycling 的容差与副作用

朴素方案每帧对每个有效接触重新计算 manifold 和材料。 如果相对姿态变化很小，重做全部工作可能浪费。 当前回收优化允许在保护条件满足时复用已有 manifold。

进入回收判断之前，仍会更新两个刚体的求解索引和逆质量。 这是必要的：睡眠迁移和稠密数组交换可能改变索引。

主要条件为：

1. 两端都不是本轮标记的 fast 物体。
2. 世界回收距离大于零。
3. 缓存相对姿态有效。
4. 接触具有回收标志；创建时要求两刚体都开启回收。
5. 两刚体相对缓存的世界旋转余弦都超过阈值。
6. 相对平移与相对转动的合成指标严格小于容差。

令 d 为相对姿态平移变化长度，单位 m；R 为两端非静态最大 extent 的最大值，单位 m。 令 $q_r.s$ 为相对旋转变化的正弦分量，无量纲。 代码比较：

$$d + R|q_r.s| < \tau.$$

默认 touching 接触 $\tau = 0.05\ \mathrm{m}$。 non-touching 接触使用 $\min(0.05, 0.02) = 0.02\ \mathrm{m}$，避免错过刚开始的接触。

取 $d = 0.01\ \mathrm{m}$、$R = 0.5\ \mathrm{m}$、$|q_r.s| = 0.02$：

$$d + R|q_r.s| = 0.01 + 0.5 \times 0.02 = 0.02\ \mathrm{m}.$$

它通过 touching 的距离条件，却不通过 non-touching 条件，因为比较是严格小于。 世界旋转条件另要求 `minCos > 0.98`，不能仅检查两物体相对姿态不变。 例如两个物体一起转过 20°，相对姿态可不变，但世界旋转条件会拒绝复用。

成功时，代码用旋转后的旧锚点临时计算新的 separation，并更新法向速度、恢复相关速度和 `persisted`。 不能说它重新生成了法线或把新锚点写回了 manifold；这不是实际代码步骤。 缓存姿态只在真正重新计算前刷新，所以容差约束的是自上次刷新以来的累计变化。

必须公开说明副作用：

- 成功回收直接 `continue`，跳过 `b2UpdateContact`。
- 因而材料参数刷新也被跳过。
- 位于该函数里的 `PreSolve` 回调同样会被跳过。
- 当前条件没有“开启 PreSolve 自动禁止回收”的分支。

如果业务依赖逐次更新材料或逐帧执行回调，必须理解并管理这条路径。 它是受阈值控制的近似优化，不应承诺与关闭回收时逐位相同。

实验：同一输入分别使用回收距离 0 和默认值。 记录 recycled contact 数、窄阶段耗时、最大穿透和事件差异。 还应安排材料修改和 PreSolve 计数实验，而不只观察画面是否相似。

17.10 自测与参考答案

1. 1,000 个形状的全配对检查次数是多少？

2. 同正组号且双方 mask 为零时，形状过滤是否允许？
3. 插入例中靠近 A 的新叶，候选最小代价是多少？
4. 旋转例的旧、新局部周长分别是多少？
5. non-touching 回收指标等于 0.02 m，默认条件能通过吗？
6. 新 `ID12` 没有旧匹配点时，热启动冲量是多少？
7. 树查询访问数减少，是否就证明整帧更快？

参考答案：

1. $1000 \times 999/2 = 499500$ 次。
2. 这个形状过滤函数允许，但其他碰撞限制仍可拒绝。
3. 11 m；把新叶移到另一手算位置后，B 候选为 16 m。
4. 20 m 与 8 m。
5. 不能，源码使用严格小于。
6. 0。
7. 不能，还要计算维护、重建、候选处理及其他阶段成本。

17.11 源码导航

主题	相对源码链接与入口行号
分组与掩码	`shape.h`，`b2ShouldShapesCollide`：122
形状 margin	`shape.c`，`b2ComputeShapeMargin`：36
配对过滤与三树查询	`broad_phase.c`，172、333
树后台重建	`broad_phase.c`，`b2UpdateTreesTask`：401
插入代价与旋转	`dynamic_tree.c`，164、315、602
查询与重建	`dynamic_tree.c`，1085、1433、1879
GJK 缓存	`distance.c`，168、206、424
点 ID 匹配	`contact.c`，`b2UpdateContact`：525
回收条件	`physics_world.c`，`b2CollideTask`：449，重点 507–585
树实验入口	`test_dynamic_tree.c`，329、398、449

返回 README · 下一章：岛、睡眠与数据布局